Week 2: Robot Missions

Last week you drove the robot with commands. This week it drives in **maze levels** and uses **sensors** to decide what to do, like a real autonomous robot. Each time you finish a mission, MATLAB gives you a **mission code**. Type it in the Week 2 answer sheet (a Word file).

What you hand in

Open `Week2_Answer_Sheet.docx`. Type your name, every **mission code** exactly as MATLAB prints it, and your answers to the exit questions. Save it and upload it to the course page before you leave.

Getting started again (first 5 minutes)

	Do this	You should see
1	Start **MATLAB R2025a**.	The Command Window (»).
2	In the **Current Folder** / **Files** panel, open your `TurtleRobot` folder from last week. *Lost it?* Download it again from the course page, or from `https://github.com/USERNAME/ENGR1701-Turtle-Robot` (green **Code** button → **Download ZIP**), and unzip it.	`Turtle.m`, `Week2_Robot_Missions.m`, ...
3	Double-click `Week2_Robot_Missions.m`.	The file opens in the Editor.
4	Click inside the **WARM-UP** section and press `Ctrl`+`Enter` (Mac: `Cmd`+`Enter`).	A robot in a hallway, and sensor values in the Command Window.

Reminder: how to work in the files

- The files are split into **sections**. A section starts with a line that begins with `%%`. Click inside a section (it turns light yellow) and press `Ctrl`+`Enter`, or click **Run Section** on the **Editor** tab. Only that section runs.
- **Do not** press the big green **Run** button (`F5`). It runs the whole file and stops at the first unfinished task.
- Green lines starting with `%` are **comments**: instructions for you. MATLAB ignores them.
- Do the sections **in order**. Type your code on the empty lines under "your code here."
- A robot stuck in a loop that runs forever? Click in the Command Window and press `Ctrl`+`C`.

Red text means an error. That's normal!

Every programmer sees errors all day. MATLAB tells you *which line* and *what* went wrong. Read the message, fix that line, and run the section again.

If you see...	It usually means...
`Unrecognized method, property, or field 'foward' for class 'Turtle'`	A command is misspelled, or has the wrong capital letters. MATLAB is case sensitive: `left` works, `Left` does not.
`Unrecognized function or variable 't'`	You skipped `t = Turtle();` or ran a section out of order.
`Unrecognized function or variable 'fifty'`	You typed a word where MATLAB needs a number or a variable you created earlier.
`Invalid expression` / `Parenthesis is not balanced`	A missing or extra `( )`. Every `(` needs a `)`.
`CRASH! The robot hit a wall...`	Your robot drove into a wall. Count the cells again or use a sensor.
`Stopped: ...Is your loop running forever?`	Your `while` loop never ends. Check that the robot moves inside the loop.
`TRY AGAIN: ...`	The code ran, but the result isn't right yet. Read the hint after it.

Robot command cheat sheet

Your robot is a variable called `t`. Every command starts with `t.` (t, then a dot).

Command	What it does
`t = Turtle();`	Make a new robot on an empty canvas (Week 1).
`t = Turtle("maze");`	Make a new robot in a level: `"hallway"`, `"corner"`, `"stairs"`, `"tunnel"`, `"mystery"`, `"maze"`.
`t.forward(100)` / `t.backward(50)`	Drive forward / backward. In mazes the number is cells.
`t.left(90)` / `t.right(45)`	Turn left / right by an angle in degrees. `t.left()` turns 90.
`t.penUp()` / `t.penDown()`	Stop / start drawing while moving.
`t.setColor("red")`	Pen color: red, blue, green, orange, purple, black, ...
`t.setWidth(4)` / `t.setSpeed(10)`	Thicker pen / faster robot (1 = slow, 10 = fast).
`t.setRobotColor("blue")`	Change the color of the robot itself (not the pen).
`t.wallAhead()`, `t.wallLeft()`, `t.wallRight()`	Sensors: `true` if there is a wall there.
`t.distanceAhead()`	Sensor: number of free cells in front of the robot.
`t.atGoal()`	Sensor: `true` if the robot is on the goal.
`t.report()`	Print how many commands the robot used.
`checkMission(t, "square")`	Check your work. A PASS gives you a mission code.

Three ideas you will use

Repeat N times

```
for k = 1:4
    t.forward(100)
    t.left(90)
end
```

Repeat while true

```
while ~t.wallAhead()
    t.forward(1)
end
```

~ means **NOT**.

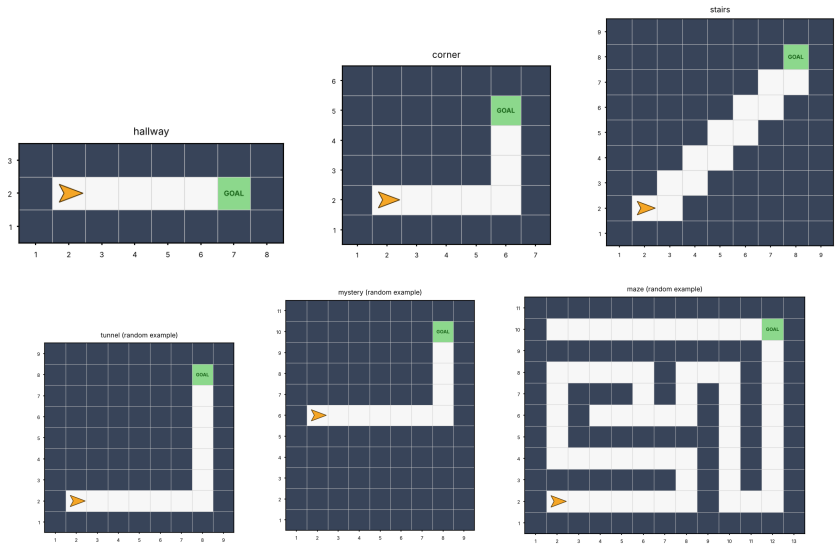
Decide

```
if t.wallLeft()
    t.right()
else
    t.left()
end
```

Week 2 – Robot missions

File: `Week2_Robot_Missions.m`.

In maze levels, **dark cells are walls**, the **green cell is the goal**, and one step is one cell. Turns are always 90 degrees. If the robot hits a wall, it **crashes** (turns red): fix your code and run the section again.



Mission	What to do
Warm-up	Run it and read what each **sensor** says.
1 Hallway	Drive straight to the goal.
2 Corner	Drive, turn, drive. Count cells first.
3 Stairs	Write *one* step inside a **for loop**, then repeat it for every step.
4 Tunnel	The length changes every run. Use a **while loop** and `t.wallAhead()`. Your code is tested on 3 random tunnels.
5 Mystery	The tunnel turns left *or* right. Use **if** / **else** with `t.wallLeft()`. Tested 3 times.
6 Final boss	Escape a random maze with the **right-hand rule**: keep your right hand on the wall. Tested on 3 mazes.
Bonus A, B	Use as few commands as possible; draw your own maze and test your code on it.

Think like a robot engineer

In Missions 1–3 you know the map, so you can plan every move. In Missions 4–6 the map changes every time, just like the real world. A real robot can't count cells it has never seen. It has to use its **sensors** and **rules** (loops and if-statements) that work in any situation. That is the main idea of robot autonomy.